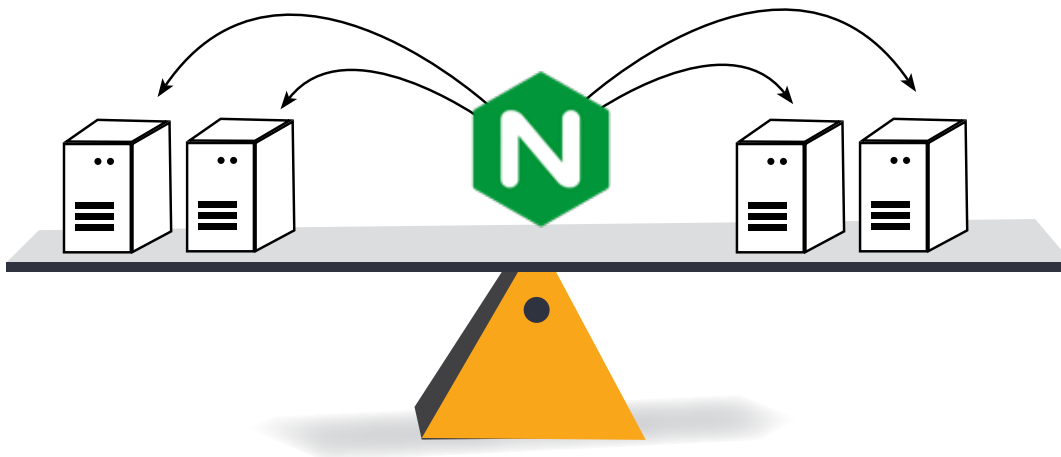
Building an Advanced Load Balancer with NGINX

To build an advanced load balancer with NGINX we need a comprehensive approach that includes installation, configuration, optimisation, and ongoing management. This guide details the steps and considerations for setting up a robust NGINX load balancer.



The first step in creating an advanced load balancer is to install NGINX on the server that will serve as the load balancer. This is typically done through the package manager of your operating system. For instance, on a Debian-based system like Ubuntu, you can execute the following commands:

```
bash
sudo apt update
sudo apt install nginx
```

For Red Hat-based systems like CentOS, you can use:

```
bash
sudo yum install epel-release
sudo yum install nginx
```

After installation, you can start NGINX with…

```
sudo systemctl start nginx
```

…and enable it to run on boot with:

```
sudo systemctl enable nginx
```

Basic load balancer configuration

The next step is to configure NGINX to distribute incoming traffic across your backend servers. You'll need to edit the NGINX configuration files, which are typically located in */etc/nginx/*. The primary configuration file is */etc/nginx/nginx.conf*, and additional server blocks (virtual hosts) can be defined in */etc/nginx/conf.d/*.

Here's a basic example of a load balancer configuration:

```
nginx
http {
    upstream myapp {
        server server1.example.com;
        server server2.example.com;
        server server3.example.com;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://myapp;
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection 'upgrade';
            proxy_set_header Host $host;
            proxy_cache_bypass $http_upgrade;
        }
    }
}
```